NOTE You can read more about why I generally don't unit-test my controllers in my blog article "Should you unit-test API/MVC controllers in ASP.NET Core?" at http://mng.bz/YqMo.

So what about minimal API endpoints? There's both good news and bad news here. On one hand, minimal API endpoints are simple lambda functions, so you can unit-test them, but these tests also suffer from many drawbacks:

- You must write your endpoint handlers as static or instance methods on a class, not as lambda methods or local functions, so that you can reference them from the test project.
- You are testing only the execution of the endpoint handler, outside any filters applied to the endpoint or route group that execute in the real app.
- You are not testing model-binding or result serialization—two common sources of errors in practice.
- If your endpoint is simple, as it should be, there's not much to test!

I find unit tests for minimal APIs to be overly restrictive and limited in value, so I avoid them, but you can see an example of a minimal API unit test in the source code for this chapter.

NOTE I haven't discussed Razor Pages much in this section, as they suffer from many of the same problems, in that they are dependent on the supporting infrastructure of the framework. Nevertheless, if you do wish to test your Razor Page `PageModel`, you can read about it in Microsoft's "Razor Pages unit tests in ASP.NET Core" documentation: http://mng.bz/GxmM.